Develop Natural Language Solutions in Azure

Language, speech, and translation with Foundry Tools

Rick Beerendonk
rick@oblicum.com

glasspaper oblicum

© Rick Beerendonk

What You'll Learn

- Pull meaning out of text: language, sentiment, entities, key phrases, PII
- Turn speech into text and text into speech
- Decide between a purpose-built service and a large model
- Build a spoken conversation that reacts in real time
- Translate written and spoken language

© Rick Beerendonk

Problem: Human Language Is Not Data

A support mailbox receives 4,000 messages a week.

- Which are angry? Which mention a competitor? Which contain a customer's phone number?
- ❌ A database cannot answer any of that with a `WHERE` clause
- ❌ Reading them by hand does not scale

Need: services that convert language into structured facts you can filter, route, and store.

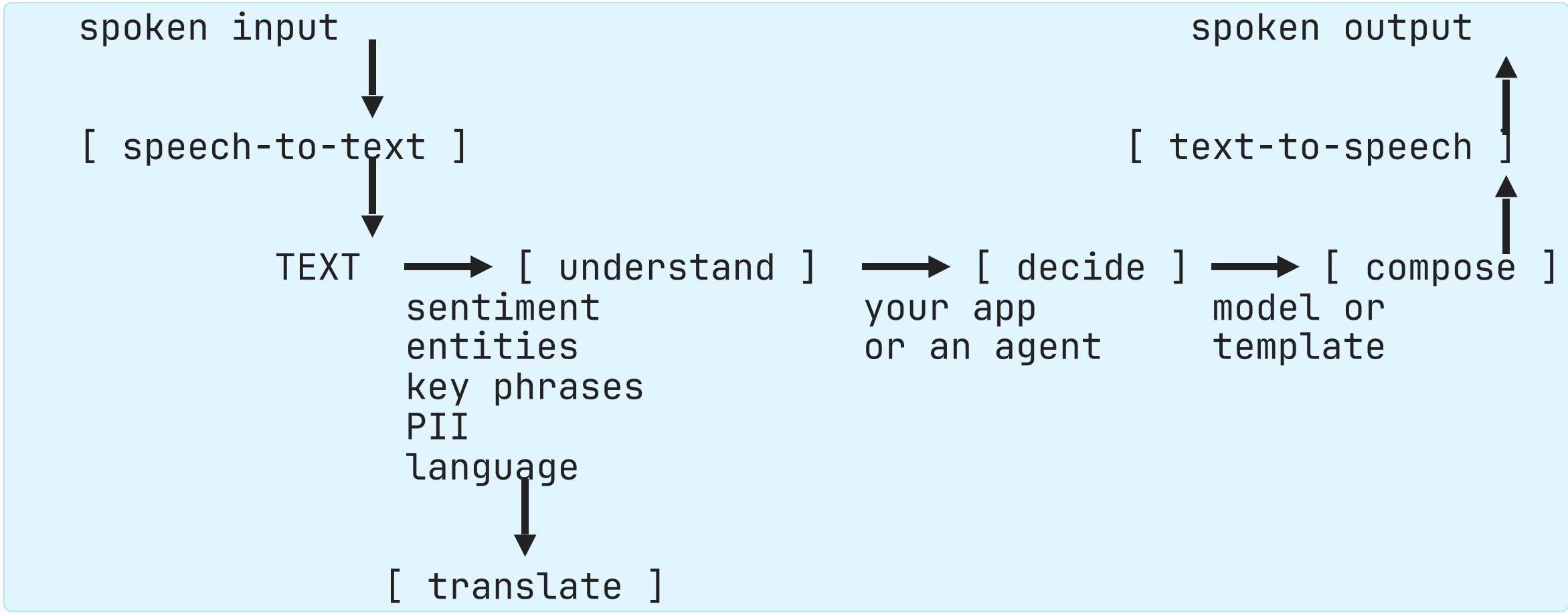
That is the whole point of this presentation: **language in, data out.**

glasspaper oblicum

© Rick Beerendonk

The Pipeline

Every solution in this presentation is a few of these boxes in a row.



Voice Live is this whole loop, running continuously, in under a second.

glasspaper oblicum

© Rick Beerendonk

Decoding the Jargon

Term	In plain words
NER	Named Entity Recognition — find the names of people, places, dates
Entity linking	Also say *which* Paris — by linking to a knowledge base entry
Key phrases	The handful of noun phrases the text is actually about
PII	Personally Identifiable Information — emails, phone numbers, ID numbers
Redaction	Replacing those values with `*****` before you store or send the text
STT / TTS	Speech-to-text (listening) / text-to-speech (talking)
Neural voice	A model-generated voice, e.g. `en-US-JennyNeural`
SSML	XML that tells the voice where to pause, and in which style to speak
Diarization	Marking *who* said each sentence when several people speak
VAD	Voice Activity Detection — deciding when the person stopped talking

© Rick Beerendonk

Purpose-Built Service or Large Model?

Both can read text. They fail in different ways.

Ask yourself	Language / Speech service	Generative model
Do I need the *same* answer every time?	Yes — fixed categories, scores	No — wording varies
Do I need an open-ended answer?	No	Yes
Do I need a confidence score?	Yes, per result	Not really
Cost and latency	Low and predictable	Higher, varies with length

Rule of thumb: if you can write the list of possible answers in advance, use the purpose-built service.

glasspaper oblicum

© Rick Beerendonk

Azure Language

glasspaper



© Rick Beerendonk

TextAnalyticsClient

Package: `pip install azure-ai-textanalytics`

```python
from azure.ai.textanalytics import TextAnalyticsClient
from azure.core.credentials import AzureKeyCredential
from azure.identity import DefaultAzureCredential

# Key-based
client = TextAnalyticsClient(endpoint="https://{resource}.services.ai.azure.com",
                             credential=AzureKeyCredential("KEY"))

# Recommended (production)
client = TextAnalyticsClient(endpoint="https://{resource}.services.ai.azure.com",
                             credential=DefaultAzureCredential())
```

Constraints: max 5,120 characters per document; documents per request depend on the feature — 5 for NER and PII, 10 for sentiment and key phrases, 1,000 for language detection

glasspaper oblicum

© Rick Beerendonk

Language Detection

```
result = client.detect_language(["Hello world", "Bonjour le monde"])
for doc in result:
    lang = doc.primary_language
    print(f"{lang.name} ({lang.iso6391_name}) – confidence: {lang.confidence_score}")
```

Edge cases:

- Multilingual text → returns dominant language with lower confidence
- Unparseable text → `name = "(Unknown)"`, `confidence_score = 0`

glasspaper oblicum

© Rick Beerendonk

Sentiment Analysis

[EXAM]

```
result = client.analyze_sentiment(["I love this service!", "The wait was too long."])
for doc in result:
    print(f"Overall: {doc.sentiment}")
    print(f"  positive={doc.confidence_scores.positive:.2f}")
    for sent in doc.sentences:
        print(f"  Sentence: '{sent.text}' → {sent.sentiment}")
```

Sentiment logic:

- All neutral → `neutral`
- Includes positive + neutral → `positive`
- Includes negative + neutral → `negative`
- Positive + negative both present → `mixed`

Lifecycle: sentiment analysis retires from Azure Language on 31 March 2029 — build new projects on a Foundry model

glasspaper oblicum

© Rick Beerendonk

Entity Recognition and Linking

[EXAM]

Named Entity Recognition (NER):

```
result = client.recognize_entities(["Microsoft was founded by Bill Gates in 1975."])
for doc in result:
    for entity in doc.entities:
        print(f"{entity.text} ({entity.category}) — {entity.confidence_score:.2f}")
```

Categories: Person, Location, DateTime, Organization, Address, Email, URL

Entity Linking:

```
result = client.recognize_linked_entities(["I visited Paris last summer."])
for doc in result:
    for entity in doc.entities:
        print(f"{entity.name} — {entity.data_source} — {entity.url}")
```

Entity linking uses Wikipedia as knowledge base; disambiguates same-name entities by context (e.g., "Venus" → planet vs. mythology); it retires on 1 September 2028 — new projects use NER or a Foundry model

glasspaper oblicum

© Rick Beerendonk

Key Phrase Extraction

```
result = client.extract_key_phrases([
    "Microsoft launched its new AI product in Seattle this week."
])
for doc in result:
    print(doc.key_phrases)  # ['new AI product', 'Microsoft', 'Seattle']
```

Lifecycle: key phrase extraction retires from Azure Language on 31 March 2029 — build new projects on a Foundry model

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 1

Pause here and complete Lab 1, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Personally Identifiable Information (PII)

[EXAM]

Use PII recognition to find and redact sensitive details before storage, indexing, or model input.

```
result = client.recognize_pii_entities([
    "Contact Rick at rick@example.com or 555-0100."
])

for doc in result:
    print(doc.redacted_text)
    for entity in doc.entities:
        print(f"{entity.text} ({entity.category})")
```

The response provides:

- `redacted_text` with detected values replaced
- Entity text, category, offset, length, and confidence score
- Categories such as person, email, phone number, address, and government ID

glasspaper oblicum

© Rick Beerendonk

PII Redaction

[EXAM]

What masking actually does — the sensitive value is **removed**, not hidden alongside the original. The redaction policy decides what replaces it: `characterMask` (the default) writes `*` for every character, `entityMask` writes the **category label**:

```
Input:          Contact Rick at rick@example.com or 555-0100.
characterMask: Contact **** at ********************* or ********.
entityMask:    Contact [PERSON_1] at [EMAIL_1] or [PHONENUMBER_1].
```

There is no overarching `Contact` category. Detection uses specific categories (person, email, phone number), so a policy that must treat phone numbers differently from names can do so.

Pipeline pattern: detect → review policy → redact or tokenize → persist the safe representation.

Do not treat detection as authorization: combine PII analysis with access controls, encryption, retention policies, and audit logging.

glasspaper oblicum

© Rick Beerendonk

Content Safety — Text Analysis

[EXAM]

Check user-generated text **before** it is published.

```python
from azure.ai.contentsafety import ContentSafetyClient
from azure.ai.contentsafety.models import AnalyzeTextOptions

request = AnalyzeTextOptions(text=comment)
response = client.analyze_text(request)

for item in response.categories_analysis:
    print(item.category, item.severity)   # e.g. SelfHarm 4
```

Element	Value
Options class	`AnalyzeTextOptions`, with the input passed as the named `text` parameter
Client method	`client.analyze_text(request)` — the synchronous `ContentSafetyClient`
Result	Per-category severity: hate, sexual, violence, self-harm

Content Safety is not PII detection: it classifies harmful-content severity; it does not find or mask personal data.

© Rick Beerendonk

Retrieval Practice — Lab 2

Pause here and complete Lab 2, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Guardrails and Responsible AI

A severity score only helps when something acts on it.

Action	What happens to the content
Block	It is rejected and does not continue
Annotate	It continues, with the severity reported as metadata

- **Check input and output:** unsafe content can arrive in a prompt or be generated in the answer
- **Use Block** when the content must not continue
- **Transparency:** tell users how their data is processed and used — one of the six responsible AI principles

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 3

Pause here and complete Lab 3, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Private Networking for a Language Resource

[EXAM]

Analyzing confidential **on-premises** documents:

1. Deploy the **Language resource**
2. Add a **private endpoint** — the service gets a private IP inside your virtual network
3. Connect from on-premises over **VPN or ExpressRoute**

Alternative	Why it fails
Public endpoint	The service stays reachable over the internet
VPN or ExpressRoute alone	Without a private endpoint the service is not private
Managed identity	Controls authentication, not network routing

Identity answers “who”; networking answers “from where.”

© Rick Beerendonk

Language MCP Server

glasspaper oblicum

© Rick Beerendonk

Language MCP Endpoint and Tools

[EXAM]

Endpoint:

`https://{foundry-resource-name}.cognitiveservices.azure.com/language/mcp?api-version=2025-11-15-preview`

Capabilities exposed as MCP tools:

Tool	Capability
Named Entity Recognition	People, places, orgs, dates, quantities
Sentiment Analysis	Positive/negative/neutral + aspect-level opinions
Summarization	Concise summaries of long text
Key Phrase Extraction	Main concepts and key phrases
PII Redaction	Detects/redacts names, addresses, phone numbers
Language Detection	Identifies language
Text Analytics for Health	Medical entities (diagnoses, medications, symptoms)
Conversational Language Understanding	Intent/entity extraction from custom trained model
Custom Question Answering	Returns answers from a knowledge base

Lifecycle: Sentiment Analysis, Summarization, Key Phrase Extraction, Conversational Language Understanding, and Custom Question Answering retire from Azure Language on 31 March 2029.

glasspaper oblicum

© Rick Beerendonk

Language MCPTool in Code

```python
from azure.ai.projects.models import MCPTool

mcp_tool = MCPTool(
    server_label="azure-language",
    server_url="https://{foundry-resource-name}.cognitiveservices.azure.com/language/mcp"
               "?api-version=2025-11-15-preview",
    require_approval="always",
)
# Optionally restrict: mcp_tool.allowed_tools = ["extract_named_entities_from_text"]
```

Client pattern:

```python
openai_client = project_client.get_openai_client()
response = openai_client.responses.create(
    input=[{"role": "user", "content": user_prompt}],
    extra_body={"agent_reference": {"name": "Text-Analysis-Agent", "type": "agent_reference"}},
)
print(response.output_text)
```

Multi-task prompts: Agent can call multiple tools in one turn (e.g., NER + sentiment)

glasspaper oblicum

© Rick Beerendonk

Speech with Generative AI

glasspaper oblicum

© Rick Beerendonk

Speech-to-Text Models (OpenAI Audio API)

```python
from openai import AzureOpenAI

client = AzureOpenAI(
    azure_endpoint=FOUNDRY_ENDPOINT, api_key=FOUNDRY_KEY,
    api_version="2025-04-01-preview"
)

audio_file = open("speech.mp3", "rb")
transcription = client.audio.transcriptions.create(
    model=MODEL_DEPLOYMENT,
    file=audio_file,
    response_format="text"
)
print(transcription)
```

Models: `gpt-transcribe`, `whisper`; in preview: `gpt-4o-transcribe`, `gpt-4o-mini-transcribe`, `gpt-4o-transcribe-diarize`
v1 route: `/openai/v1/audio/…` is still preview and needs `api-version=preview`

glasspaper oblicum

© Rick Beerendonk

Text-to-Speech Models (OpenAI Audio API)

```python
with client.audio.speech.with_streaming_response.create(
    model=MODEL_DEPLOYMENT,
    voice="alloy",
    input="This speech was AI-generated!",
    instructions="Speak in an upbeat, excited tone.",
) as response:
    response.stream_to_file("speech_output.mp3")
```

Models (preview): `gpt-4o-mini-tts`, `tts`, `tts-hd`
Key parameters: `voice` (e.g., `"alloy"`), `input` (text), `instructions` (tone/style; `gpt-4o-mini-tts` only)

glasspaper oblicum

© Rick Beerendonk

Azure Speech SDK

glasspaper



© Rick Beerendonk

Choosing a Speech Capability

[EXAM]

Requirement	Choose
Transcribe a **live** phone call with low latency	**Real-time speech-to-text** on the streaming audio
Transcribe recordings that are already complete	Batch transcription
Two-way spoken interaction with low latency	**Real-time speech-to-text in, text-to-speech out**
Produce audio from text	Text-to-speech
Change the language of what was said	Speech translation

Batch transcription waits for the recording to finish, so it cannot serve a live transcript and it breaks turn-taking. Text-to-speech generates audio instead of transcribing it, translation changes language rather than producing a transcript, and embeddings support similarity search rather than conversation.

© Rick Beerendonk

SpeechConfig and Speech-to-Text

[EXAM]

Package: `azure-cognitiveservices-speech` (SDK ≥ 1.48.2)

```python
import azure.cognitiveservices.speech as speech_sdk

speech_config = speech_sdk.SpeechConfig(subscription="KEY", endpoint="ENDPOINT")

audio_config = speech_sdk.audio.AudioConfig(filename="audio.wav")
# or: use_default_microphone=True

recognizer = speech_sdk.SpeechRecognizer(speech_config=speech_config,
                                         audio_config=audio_config)
result = recognizer.recognize_once_async().get()

if result.reason == speech_sdk.ResultReason.RecognizedSpeech:
    print(result.text)
```

`ResultReason` values: `RecognizedSpeech`, `NoMatch`, `Canceled`
`SpeechRecognitionResult` properties: `Duration`, `OffsetInTicks`, `Properties`, `Reason`, `ResultId`, `Text`

Continuous recognition: `start_continuous_recognition()` with the `recognizing` (partial) and `recognized` (final) events — for ongoing or live audio; `recognize_once_async()` stops after one utterance

© Rick Beerendonk

Text-to-Speech (SDK)

[EXAM]

```
audio_config = speech_sdk.audio.AudioOutputConfig(use_default_speaker=True)
speech_synthesizer = speech_sdk.SpeechSynthesizer(
    speech_config=speech_config, audio_config=audio_config
)
result = speech_synthesizer.speak_text_async("Hello world").get()

if result.reason == speech_sdk.ResultReason.SynthesizingAudioCompleted:
    print("Done")
```

Audio format:

```
speech_config.set_speech_synthesis_output_format(
    speech_sdk.SpeechSynthesisOutputFormat.Riff24Khz16BitMonoPcm
)
```

Voice:

```
speech_config.speech_synthesis_voice_name = 'en-US-Brian:DragonHDLatestNeural'
```

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 4

Pause here and complete Lab 4, then check its solution.

glasspaper oblicum

© Rick Beerendonk

SSML

[EXAM]

```
ssml = """
<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis"
       xmlns:mstts="https://www.w3.org/2001/mstts" xml:lang="en-US">
  <voice name="en-US-JennyNeural">
    <mstts:express-as style="cheerful">
      Welcome to our service!
      <break time="500ms"/>
      How can I help you today?
    </mstts:express-as>
  </voice>
</speak>
"""
result = speech_synthesizer.speak_ssml_async(ssml).get()
```

SSML capabilities: speaking style, pauses (`<break>`), phonemes (`<phoneme>`), prosody, say-as rules, recorded audio insertion

Requirement	Use
Pronounce a technical term or product name precisely	`<phoneme>`
Read dates, numbers, or currency correctly	`<say-as>`
Change rate, pitch, or volume	`<prosody>`
A brand-specific voice	Custom voice — requires training a model

Least effort for pronunciation is `<phoneme>`, not a custom voice (formerly custom neural voice).

glasspaper oblicum

© Rick Beerendonk

Custom Speech

[EXAM]

Train a custom speech-to-text model when base recognition mishears **product names, jargon, or accents**, then deploy it to a custom endpoint.

Workflow: create a project → upload audio and transcripts → train → evaluate → deploy to an endpoint

```
speech_config.endpoint_id = "<custom endpoint ID>"
```

The SDK selects the deployed model by its **endpoint ID**. REST requests that create models and endpoints take the **Custom Speech project ID** in the `project` property — not the project URL and not the endpoint URL.

When a custom model expires:

Behavior	Correct?
Requests fall back to the latest **base model** for that locale	**Yes**
Requests start returning 4xx errors	No — only **batch transcription** fails this way
The expired model keeps serving until removed	No
The model is deleted automatically	No

Consequence: recognition keeps working, but domain accuracy silently drops. Monitor expiry dates and retrain before them.

glasspaper oblicum
© Rick Beerendonk

Retrieval Practice — Lab 5

Pause here and complete Lab 5, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Speech MCP Server

glasspaper oblicum

© Rick Beerendonk

Speech MCP Endpoint and Tools

Endpoint:

`https://{foundry-resource-name}.cognitiveservices.azure.com/speech/mcp?api-version=2025-11-15-preview`

Two tools:

Capability	Details
Speech-to-text (Recognize)	Supports WAV, MP3, OGG, FLAC, MP4, M4A, AAC; language; phrase hints; profanity filtering (masked/removed/raw)
Text-to-speech (Synthesize)	Neural voices (e.g., `en-US-JennyNeural`); WAV/MP3 output; URL returned

Storage requirement: Azure Blob Storage account required for audio I/O
SAS URL permissions required: Read and Write on the container (the official lab ticks Read, Add, Create, Write, and List)

© Rick Beerendonk

Speech MCPTool in Code

```
from azure.ai.projects.models import MCPTool

mcp_tool = MCPTool(
    server_label="azure-speech",
    server_url="https://{foundry-resource-name}.cognitiveservices.azure.com/speech/mcp"
               "?api-version=2025-11-15-preview",
    require_approval="always",
)
```

Portal connection settings:

- `foundry-resource-name`: name of the Foundry resource
- `Bearer (API key)`: key of the Foundry resource, with the key name `Ocp-Apim-Subscription-Key`
- `X-Blob-Container-Url`: SAS URL for blob container

Prompt-based customization: Voice selection (`using the voice "en-GB-SoniaNeural"`), language (`es-ES`), phrase hints, profanity filtering

© Rick Beerendonk

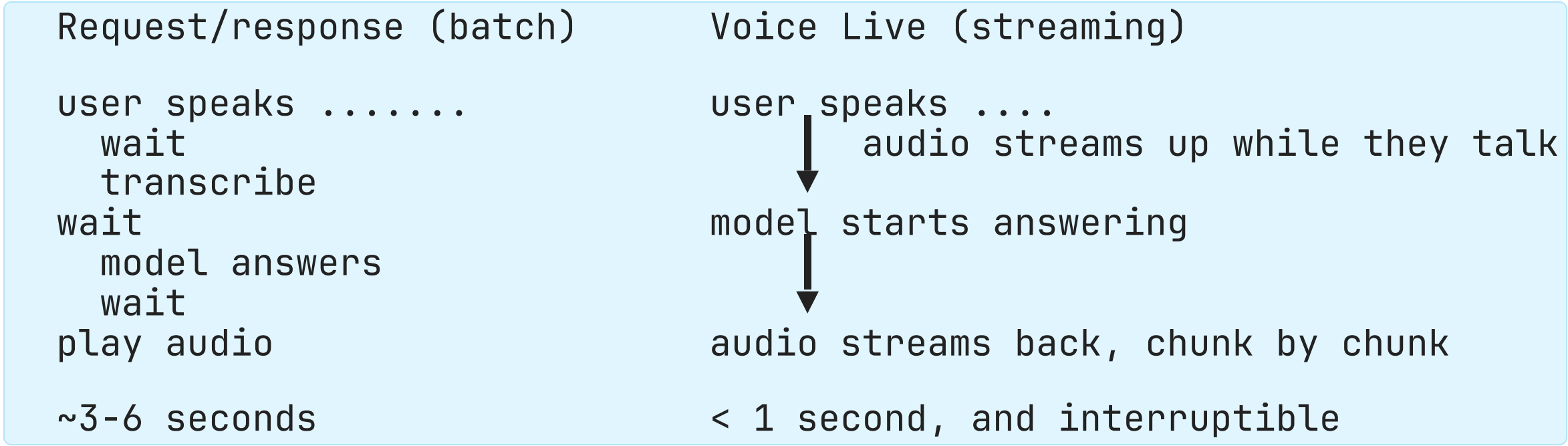
Voice Live API

glasspaper oblicum

© Rick Beerendonk

Why Real Time Is a Different Problem

Record → transcribe → think → speak takes several seconds. In a conversation, that feels broken.



Interruption is the hard part. When `input_audio_buffer.speech_started` arrives, the user began talking over the answer — stop playback immediately.

VAD decides when a turn ended, so nobody has to press a button.

glasspaper oblicum

© Rick Beerendonk

Voice Live — Real-Time Speech

[EXAM]

WebSocket-based, real-time bidirectional speech; low latency

Endpoints:

Resource	Endpoint
Foundry resource	`wss://{resource}.services.ai.azure.com/voice-live/realtime?api-version=2026-07-15`
Older resource (legacy)	`wss://{resource}.cognitiveservices.azure.com/voice-live/realtime?api-version=2026-07-15`

Authentication (recommended): Entra ID Bearer token; requires the **Cognitive Services User** and **Foundry User** (formerly Azure AI User) roles
API key auth: `api-key` header (not browser-compatible) or `api-key` query parameter

Why WebSocket: one connection carrying bidirectional streaming audio with low latency — the server-to-server choice.

Transport	Compared with WebSocket
RTMP	Not an option: built for one-way broadcast streaming
WebRTC	Also supported (preview): audio straight from a browser or mobile app, lower latency
SIP	Not an option: handles call signaling, not the audio stream itself

© Rick Beerendonk

Key Events

Client → Server:

Event	Purpose
`session.update`	Modify session settings (voice, instructions, VAD)
`input_audio_buffer.append`	Add base64-encoded audio data
`response.create`	Trigger model inference

Server → Client:

Event	Purpose
`session.updated`	Session config confirmed
`response.done`	Response generation complete
`response.audio.delta`	Audio chunk for playback
`input_audio_buffer.speech_started`	User started speaking — cancel playback
`response.audio_transcript.done`	Transcript of AI response ready

© Rick Beerendonk

Session Configuration

```
{
  "type": "session.update",
  "session": {
    "modalities": ["text", "audio"],
    "voice": { "type": "openai", "name": "alloy" },
    "instructions": "You are a helpful voice assistant.",
    "input_audio_format": "pcm16",
    "output_audio_format": "pcm16",
    "input_audio_sampling_rate": 24000,
    "turn_detection": {
      "type": "azure_semantic_vad",
      "threshold": 0.5,
      "prefix_padding_ms": 420,
      "silence_duration_ms": 500
    },
    "input_audio_noise_reduction": { "type": "azure_deep_noise_suppression" },
    "input_audio_echo_cancellation": { "type": "server_echo_cancellation" }
  }
}
```

Python SDK: `azure-ai-voicelive` (async-only, v1.0.0+)

glasspaper oblicum

© Rick Beerendonk

Translation

glasspaper



© Rick Beerendonk

Azure Translator — Text Translation

[EXAM]

100+ languages; package: `azure-ai-translation-text` (2.x)

```python
from azure.ai.translation.text import TextTranslationClient
from azure.core.credentials import AzureKeyCredential

client = TextTranslationClient(credential=AzureKeyCredential("KEY"),
                               endpoint="FOUNDRY_ENDPOINT")
# or use region: TextTranslationClient(credential=..., region="FOUNDRY_REGION")

# Detect supported languages
languages = client.get_supported_languages(scope="translation")

# Translate (auto-detects source language)
result = client.translate(
    body=["Hola", "こんにちは"],
    to_language=["fr", "en"]
)
for doc in result:
    print(f"Detected: {doc.detected_language.language}")
    for t in doc.translations:
        print(f"  {t.language}: {t.text}")
```

Mixed-language input: split a segment containing multiple languages into single-language segments, then translate each segment separately. Automatic detection returns **one** language per segment, and forcing a single source language mistranslates the other one.

© Rick Beerendonk

Transliteration and Speech Translation

Transliterate (script conversion):

```
result = client.transliterate(
    body=["こんにちは"],
    language="ja", from_script="Jpan", to_script="Latn"
)
# → "Kon'nichiwa"
```

Speech Translation (SDK):

```
translation_cfg = speech_sdk.translation.SpeechTranslationConfig(
    subscription="KEY", endpoint="ENDPOINT"
)
translation_cfg.speech_recognition_language = "en-US"
translation_cfg.add_target_language("fr")
translation_cfg.add_target_language("ja")

translator = speech_sdk.translation.TranslationRecognizer(
    translation_config=translation_cfg,
    audio_config=speech_sdk.AudioConfig(use_default_microphone=True)
)
result = translator.recognize_once_async().get()
print(result.translations["fr"])   # "Bonjour."
print(result.translations["ja"])   # "こんにちは。"
```


© Rick Beerendonk

Retrieval Practice — Lab 6

Pause here and complete Lab 6, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Key Takeaways

1. **TextAnalyticsClient** — language detection, NER, PII; sentiment, key phrases, and entity linking retire in 2028–2029; 5,120 characters per document, 5 to 1,000 documents per request
2. **Language MCP server** — nine Language capabilities as tools; agent decides which to use per prompt; `MCPTool` to attach
3. **OpenAI audio models** — `audio.transcriptions` with `gpt-transcribe`, `audio.speech` with `gpt-4o-mini-tts`; API version `2025-04-01-preview`
4. **Azure Speech SDK** — `SpeechConfig`, `SpeechRecognizer`, `SpeechSynthesizer`; SSML for style control
5. **Speech MCP server** — Blob Storage required for audio I/O; two tools: recognize + synthesize
6. **Voice Live API** — WebSocket real-time speech; `session.update` with VAD, noise reduction, echo cancellation
7. **Translation** — `TextTranslationClient`; `translate()` + `transliterate()`; `SpeechTranslationConfig` for spoken input
8. **Live vs recorded** — real-time speech-to-text for live audio, batch transcription only for completed recordings
9. **Private endpoint + VPN/ExpressRoute** — the network-isolation pattern

© Rick Beerendonk